Rockchip RV1106/RV1103 RT-Thread MCU SDK 快速开发指南

文档标识：RK-JC-YF-992

发布版本：V1.1.0

日期：2023-07-03

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2023 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文主要描述了RV1106/RV1103 RT-Thread MCU SDK的基本使用方法，旨在帮助开发者快速了解并使用SDK开发包。

产品版本

芯片名称	**RT-Thread版本**
RV1106/RV1103	V3.1.3

读者对象

本文档（本指南）主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	CWW	2023-04-18	初始版本
V1.1.0	SYF	2023-07-03	1. 增加如何添加编译配置 2. 增加如何添加Camera驱动编译配置选项 3. 增加配置上电时序

目录

Rockchip RV1106/RV1103 RT-Thread MCU SDK 快速开发指南

1. 服务器环境搭建
2. 获取SDK包
3. SDK目录结构
4. 编译说明
   - 4.1 查看SDK版本以及编译配置
   - 4.2 一键编译
   - 4.3 固件打包
5. 软件开发指南
   - 5.1 如何添加一个新的板级
     - 5.1.1 分辨率枚举信息
     - 5.1.2 CIS(cmos image sensor)设备注册
       - 5.1.2.1 struct gpio_desc
       - 5.1.2.2 camera_board_desc 结构体说明
       - 5.1.2.3 cif_hw_config 函数说明
       - 5.1.2.4 单摄注册
       - 5.1.2.5 双摄注册
     - 5.1.3 如何添加编译配置
   - 5.2 如何新增一个摄像头驱动
     - 5.2.1 如何添加Camera驱动编译配置选项
     - 5.2.2 寄存器序列转换
     - 5.2.3 配置支持的模式
       - 5.2.3.1 数据结构描述
     - 5.2.4 配置上电时序
   - 5.3 RT-Thread 初始化流程
     - 5.3.1 流程图
     - 5.3.2 说明
6. 注意事项

1. 服务器环境搭建

本 SDK 开发环境是在 Ubuntu 系统上开发测试。我们推荐使用 Ubuntu 18.04 的系统进行编译。其他的 Linux 版本可能需要对软件包做相应调整。除了系统要求外，还有其他软硬件方面的要求。

硬件要求：64 位系统，硬盘空间大于 20G。如果您进行多个构建，将需要更大的硬盘空间。

软件要求：Ubuntu 18.04 系统

编译 SDK 环境搭建所依赖的软件包安装命令如下：

```
# Notice: use python2 default
sudo apt-get update --fix-missing

sudo apt-get install repo git scons \
python-pip

pip install numpy
pip install configparser
```

建议使用 Ubuntu18.04 系统或更高版本开发，若编译遇到报错，可以视报错信息，安装对应的软件包。

2. 获取SDK包

请联系业务或FAE获取对应芯片的SDK Release文档。

比如：RV1106/RV1103 RT-Thread MCU SDK Release: Rockchip_RV1106_RV1103_RT-Thread_MCU_SDK_Release_XXX_CN.pdf

更新 SDK 代码的命令： `.repo/repo/repo sync -c --no-tags`

3. SDK目录结构

目录路径	说明
build.sh	SDK编译脚本 软连接到 project/build.sh
project	编译配置目录
prebuilts/gcc/linux-x86/riscv64	交叉工具链
rt-thread	RT-Thread 代码
rt-thread/bsp/rockchip/common/hal	RT-Thread 的 HAL 层代码 (Hardware Abstract Layer)
rt-thread/applications/battery-ipc	RT-Thread 电池IPC快启的应用参考代码
rt-thread/bsp/rockchip/rv1106-mcu	RV1106/RV1103 板级配置
output/image	固件输出目录
docs	文档目录

4. 编译说明

- 选择编译配置

根据不同的应用配置选择不同的板级。

`./build.sh lunch`

```
$ ./build.sh lunch

You're building on Linux
Lunch menu...pick a combo:
naming rules:[board]-[sensor]-[lightsensor]
命名规则:[板级]-[传感器型号]-[光敏类型]

-----------------------------------------------------
0.rv1106_evb-SC3338-ADC
      Board(板级) :rv1106_evb
   Sensor(传感器) :SC3338
LightSensor(光敏) :ADC
-----------------------------------------------------

-----------------------------------------------------
1.rv1106_evb-SC4336-ADC
      Board(板级) :rv1106_evb
   Sensor(传感器) :SC4336
LightSensor(光敏) :ADC
-----------------------------------------------------

-----------------------------------------------------
2.rv1106_evb-dual_cam_SC3338_SC3338-ADC
      Board(板级) :rv1106_evb
   Sensor(传感器) :dual_cam_SC3338_SC3338
LightSensor(光敏) :ADC
```

```
-----------------------------------------------------

Which would you like? [0]:
```

4.1 查看SDK版本以及编译配置

```
./build.sh info

$ ./build.sh info
**************************************
Check [OK]: makeinfo --version
**************************************
Check [OK]: make -v
Build SDK version: rv1106_rv1103_rt-thread_mcu_v1.0.0_20230418.xml
Current Building Information:
Target Misc config:
[build.sh:info] Build ==> rv1106_evb-SC3338-ADC
RK_BUILD_VERSION_TYPE=RELEASE
RK_JOBS=7
RK_PROJECT_OUTPUT=/home/sdk/rtt/output/out
RK_PROJECT_OUTPUT_IMAGE=/home/sdk/rtt/output/image
RK_PROJECT_TOP_DIR=/home/sdk/rtt/project
```

4.2 一键编译

```
./build.sh lunch     # Select reference board level
./build.sh           # One-click automatic compilation
```

4.3 固件打包

```
./build.sh firmware
```

固件输出目录： output/image

编译后，复制output/image/rtthread.bin 到Linux 电池IPC SDK对应目录 （例如 `sysdrv/source/uboot/rkbin/bin/rv11/rv1106_hpmcu_tb_sc230ai_v1.64.bin`）。

5. 软件开发指南

5.1 如何添加一个新的板级

1. 在camera.h中实现xxxx_support_mode_id枚举。参照分辨率枚举信息
2. 实现camera_board_desc结构体和cif_hw_config函数。参照CIS设备注册

5.1.1 分辨率枚举信息

[示例]

```
enum sc3338_support_mode_id
{
    SC3338_320X240 = 0,
    SC3338_640X480,
    SC3338_1152X648,
    SC3338_2304X1296,
    SC3338_MODE_ID_MAX = SC3338_2304X1296,
};
extern const struct camera_board_desc camera_sc3338[];
```

5.1.2 CIS(cmos image sensor)设备注册

5.1.2.1 struct gpio_desc

[定义]

```
struct gpio_desc
{
    uint32_t gpio_pin;
    struct GPIO_REG *gpio_group;
};
```

5.1.2.2 camera_board_desc 结构体说明

[描述]

Sensor与板级硬件相关的配置信息

[定义]

```
struct camera_board_desc
{
    struct gpio_desc rst_gpio;
    struct gpio_desc pwdn_gpio;
    struct gpio_desc pwren_gpio;
    eCLOCK_Name mclk_id;
    char isp_subdev_name[RK_CAMERA_DEVICE_NAME_SIZE];
    char i2c_bus[RK_CAMERA_I2C_NAME_SIZE];
    uint8_t i2c_addr;
    uint8_t time_valid_delay;
    uint8_t gain_valid_delay;
    uint8_t mode_id;
    bool rst_active;
    bool pwdn_active;
    bool pwren_active;
};
```

[成员]

成员名称	描述
rst_gpio	硬件复位引脚配置, 可以根据gpio_desc的定义实现。
pwdn_gpio	低功耗模式引脚配置,可以根据gpio_desc的定义实现。
pwren_gpio	上电引脚配置,可以根据gpio_desc的定义实现。
mclk_id	通过配置0或者1选择使用mclk0 或者mclk1。
isp_subdev_name	Sensor是否作为主摄连接到ISP。主摄：RK_ISP_MAIN_SUBDEV_NAME 辅摄：RK_ISP_ASSIST_SUBDEV_NAME
i2c_bus	选择Sensor I2C总线
i2c_addr	配置Sensor I2C地址（七位地址）
time_valid_delay	曝光时间生效所需要的帧数
gain_valid_delay	曝光增益生效所需要的帧数
mode_id	选择Sensor分辨率，可以根据camera.h里面的分辨率配置
rst_active	硬件复位引脚的有效电平
pwdn_active	低功耗模式引脚的有效电平
pwren_active	上电引脚的有效电平

5.1.2.3 cif_hw_config 函数说明

data_lanes：sensor lane数

Sensor、csi_dphy、csi_host连接关系如下表

主摄与dphy硬件连接关系	dphy1.enable	dphy2.enable	csi_host_0.enable	csi_host_1.enable
lane0 < - > MIPI_CSI_RX_D0N/P	1	0	1	0
lane1 < - > MIPI_CSI_RX_D1N/P	1	0	1	0
lane2 < - > MIPI_CSI_RX_D2N/P	0	1	0	1
lane3 < - > MIPI_CSI_RX_D3N/P	0	1	0	1

5.1.2.4 单摄注册

[示例]

以SC3338为例：

```
const struct camera_board_desc camera_sc3338[] =
{
    {
        .i2c_addr = 0x30,
        .rst_gpio =
        {
            .gpio_pin = GPIO_PIN_C5,
            .gpio_group = GPIO3,
        },
        //power down, power enable gpio, config as this if sensor has these pins
        //.pwdn_gpio =
        //{
        //    .gpio_pin = GPIO_PIN_D3,
        //    .gpio_group = GPIO3,
        //},
        //.pwren_gpio =
        //{
        //    .gpio_pin = GPIO_PIN_A1,
        //    .gpio_group = GPIO1,
        //},
        .time_valid_delay = 2,
        .gain_valid_delay = 2,
        .mode_id = SC3338_1152X648,
        .isp_subdev_name = RK_ISP_MAIN_SUBDEV_NAME,
        .i2c_bus = "i2c4",
        .rst_active = 0,
        .mclk_id = CLK_REF_MIPI0,
    },
    { /* sentinel */ }
};
```

```
void cif_hw_config(void)
{
    struct dphy_board_desc *csi2_dphy = (struct dphy_board_desc
*)&csi2_dphy_board;
    struct csi2_board_desc *csi_host_0 = (struct csi2_board_desc
*)&csi2_host0_board;
    struct vicap_board_desc *vicap = (struct vicap_board_desc *)&vicap_board;

    csi2_dphy->enable = true;
    csi2_dphy->csi2_dphy1.enable = true;
    csi2_dphy->csi2_dphy1.csi_host_idx = 0;
    //sensor data lane
    csi2_dphy->csi2_dphy1.data_lanes = 2;
    csi2_dphy->csi2_dphy1.isp_subdev_name = RK_ISP_MAIN_SUBDEV_NAME;

    csi_host_0->enable = true;

    vicap->enable = true;
}
```

5.1.2.5 双摄注册

[示例]

以SC3338为例：

```
const struct camera_board_desc camera_sc3338[] =
{
    {
        .i2c_addr = 0x32,
        .rst_gpio =
        {
            .gpio_pin = GPIO_PIN_D1,
            .gpio_group = GPIO3,
        },
        .time_valid_delay = 2,
        .gain_valid_delay = 2,
        .mode_id = SC3338_2304X1296,
        .isp_subdev_name = RK_ISP_ASSIST_SUBDEV_NAME,
        .i2c_bus = "i2c4",
        .rst_active = 0,
        .mclk_id = CLK_REF_MIPI1,
    },
    {
        .i2c_addr = 0x30,
        .rst_gpio =
        {
            .gpio_pin = GPIO_PIN_C5,
            .gpio_group = GPIO3,
        },
        .time_valid_delay = 2,
        .gain_valid_delay = 2,
        .mode_id = SC3338_1152X648,
        .isp_subdev_name = RK_ISP_MAIN_SUBDEV_NAME,
        .i2c_bus = "i2c4",
        .rst_active = 0,
        .mclk_id = CLK_REF_MIPI0,
    },
    { /* sentinel */ }
};
```

```
void cif_hw_config(void)
{
    struct dphy_board_desc *csi2_dphy = (struct dphy_board_desc
*)&csi2_dphy_board;
    struct csi2_board_desc *csi_host_0 = (struct csi2_board_desc
*)&csi2_host0_board;
    struct csi2_board_desc *csi_host_1 = (struct csi2_board_desc
*)&csi2_host1_board;
    struct vicap_board_desc *vicap = (struct vicap_board_desc *)&vicap_board;

    csi2_dphy->enable = true;
    csi2_dphy->csi2_dphy1.enable = true;
    csi2_dphy->csi2_dphy1.csi_host_idx = 0;
    csi2_dphy->csi2_dphy1.data_lanes = 2;
    csi2_dphy->csi2_dphy1.isp_subdev_name = RK_ISP_MAIN_SUBDEV_NAME;
```

```
    csi2_dphy->csi2_dphy2.enable = false;
    csi2_dphy->csi2_dphy2.csi_host_idx = 1;
    csi2_dphy->csi2_dphy2.data_lanes = 2;
    csi2_dphy->csi2_dphy2.isp_subdev_name = RK_ISP_ASSIST_SUBDEV_NAME;

    csi_host_0->enable = true;
    csi_host_1->enable = false;

    vicap->enable = true;
}
```

5.1.3 如何添加编译配置

```
cd rt-thread/bsp/rockchip/rv1106-mcu/
scons --menuconfig
```

配置会保存在rt-thread/bsp/rockchip/rv1106-mcu/.config里面，需要用以下命令替换到对应的板级配置中

```
cp rt-thread/bsp/rockchip/rv1106-mcu/.config rt-thread/bsp/rockchip/rv1106-mcu/board/<board name>/defconfig
```

注意：添加编译配置时要修改board name，若board name重复，可能会编译至其他板级配置

5.2 如何新增一个摄像头驱动

5.2.1 如何添加Camera驱动编译配置选项

在rt-thread/bsp/rockchip/common/drivers/Kconfig添加如下修改

```
@@ -506,6 +506,8 @@ menu "supported camera"
         bool "Enable SC3336"
     config RT_USING_SC3338
         bool "Enable SC3338"
+    config RT_USING_OS02K10
+        bool "Enable OS02K10"
     config RT_USING_SC4336
         bool "Enable SC4336"
     config RT_USING_GC3003
```

5.2.2 寄存器序列转换

将初始化序列替换至rt-thread/bsp/rockchip/common/drivers/camera/reg.txt中

```
cd rt-thread/bsp/rockchip/common/drivers/camera
python2 reg_conver.py
```

将转化的结果复制替换到Sensor驱动中的寄存器序列

```
/*
 * Xclk 27Mhz
 * max_framerate 120fps
 * mipi_datarate per lane 225Mbps, 2lane
 * binning to 1152x648 then centered crop
 */
static const uint8_t sc3338_320x240_120fps_regs[] =
{
    0x3, 0x36, 0xe9, 0x80,
    0x3, 0x37, 0xf9, 0x80,
    0x3, 0x30, 0x1f, 0x6a,
    0x3, 0x30, 0xb8, 0x44,
    0xe, 0x32, 0x00, 0x00, 0x00, 0x01, 0x98, 0x09, 0x07, 0x03, 0x7f, 0x01, 0x40,
0x00, 0xf0,
    0x5,
};
```

注意事项：寄存器序列中不要开启出流寄存器，在stream on函数中单独开启

5.2.3 配置支持的模式

5.2.3.1 数据结构描述

struct xxxx_mode

[描述]

Sensor能支持各个模式的信息。

该结构体可根据需求增加变量。

[定义]

```
struct xxxx_mode
{
    struct v4l2_fract max_fps;
    uint16_t bus_fmt;
    uint16_t width;
    uint16_t height;
    uint16_t hts_def;
    uint16_t vts_def;
    uint16_t exp_def;
    uint32_t xvclk;
    uint32_t link_freq;
    const uint8_t *reg_list;
    const int reg_list_size;
    uint8_t hdr_mode;
} __attribute__((__packed__));
```

[成员]

成员名称	描述
.max_fps	图像帧率，denominator/numerator = fps。
.bus_fmt	Sensor输出格式。
.width	有效图像宽度。需要和sensor当前配置的width输出一致。
.height	有效图像高度。需要和sensor当前配置的height输出一致。
.hts_def	默认HTS，为有效图像宽度 + HBLANK。
.vts_def	默认VTS，为有效图像宽度 + VBLANK。
.exp_def	默认曝光时间。
.xvclk	Sensor 外部时钟频率，即mclk
.link_freq	MIPI数据传输速率（单位MHz）
*reg_list	寄存器列表
.reg_list_size	寄存器数组大小
.hdr_mode	Sensor工作模式，支持线性模式，两帧合成HDR,三帧合成HDR

[示例]

```
static const struct sc3338_mode supported_modes[] =
{
    [SC3338_320X240] = {
        .bus_fmt = MEDIA_BUS_FMT_SBGGR10_1X10,
        .width = 320,
        .height = 240,
        .max_fps = {
            .numerator = 10000,
            .denominator = 1200000,
        },
        .exp_def = 0x46,
        .hts_def = 0x0578 * 2,
        .vts_def = 0x012c,
        .xvclk = 27000000,
        .link_freq = 112500000,
        .reg_list = sc3338_320x240_120fps_regs,
        .reg_list_size = ARRAY_SIZE(sc3338_320x240_120fps_regs),
        .hdr_mode = NO_HDR,
    },
};
```

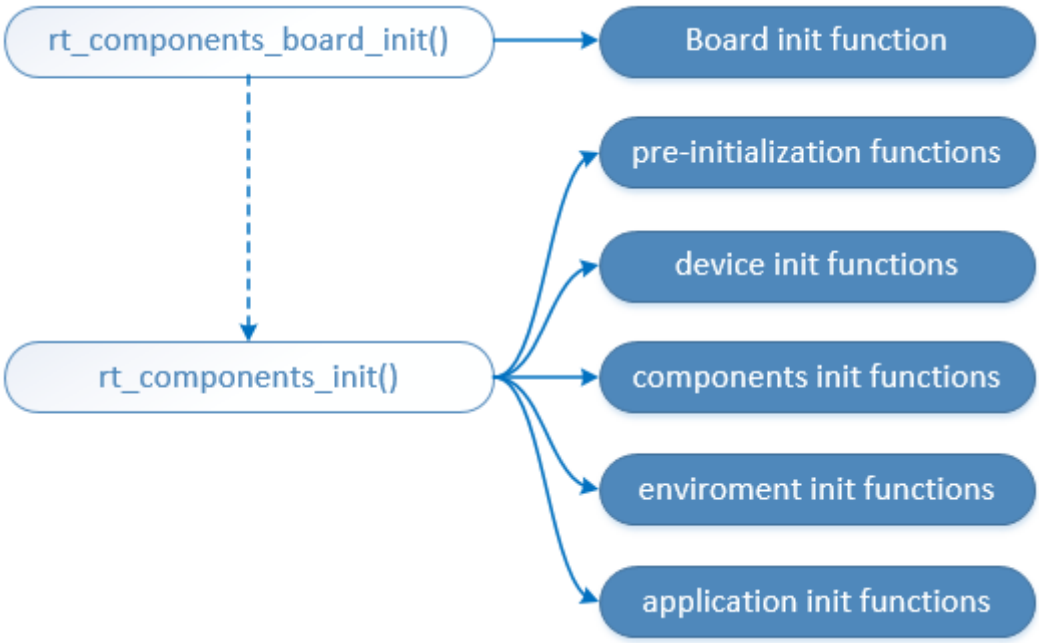
5.2.4 配置上电时序

根据sensor的datasheet，使用`HAL_DelayMs()`函数做相应的延时。

```
HAL_Status HAL_DelayMs(uint32_t ms);
```

5.3 RT-Thread 初始化流程

5.3.1 流程图



5.3.2 说明

从上图可知RT-Thread使用rt_components_board_init() 和rt_components_init()两个函数去完成六个部分的初始化。当我们需要去添加初始化内容时，我们可以使用下表所示的宏接口去实现自动初始化。初始化优先级可以参照下表：

Initialization Level	Macro interface	Description
1	INIT_BOARD_EXPORT(fn)	Board initialization
2	INIT_PREV_EXPORT(fn)	Pure software initialization
3	INIT_DEVICE_EXPORT(fn)	Initialization of peripheral drivers
4	INIT_COMPONENT_EXPORT(fn)	Component auto initialize
5	INIT_ENV_EXPORT(fn)	System Environment auto initialize
6	INIT_APP_EXPORT(fn)	Application initialization

6. 注意事项

在windows下复制源码包时，linux下的可执行文件可能变为非可执行文件，或者软连接失效导致无法编译使用。
因此使用时请注意不要在windows下复制源代码包。